ENTERPRISE DIGITAL BANKING SYSTEM

02

Database Documentation

ER diagram, relational schema, data dictionary, constraints, indexes and migration history for banking_db and notification_db.

Document	02 — Database Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Overview
2. 2. Entity Relationship Diagram
3. 3. Relational Schema (DDL)
4. 4. Data Dictionary
5. 5. Constraints and Integrity Rules
6. 6. Indexes
7. 7. Migration History
8. 8. Data Retention and Backup

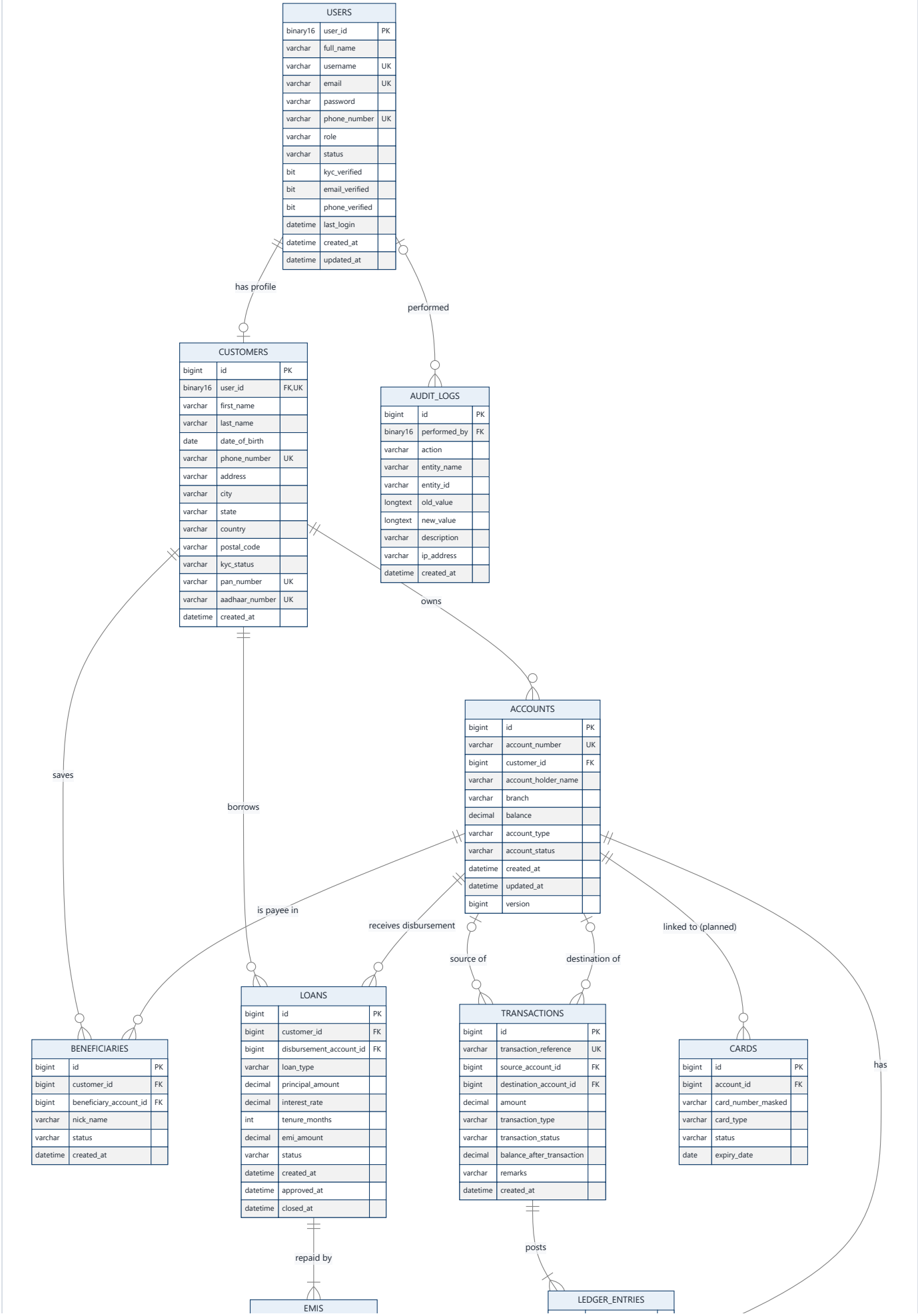
1. Overview

Item	Value
RDBMS	MySQL 8.0 (InnoDB, utf8mb4)
Schemas	`banking_db` (banking-app) · `notification_db` (notification-service) — database-per-service, no cross-schema foreign keys
Connection	`jdbc:mysql://localhost:3306/banking_db?createDatabaseIfNotExist=true`, user `${DB_USERNAME:root}`, password `${DB_PASSWORD}`
ORM	Hibernate (Spring Data JPA). Today schema is managed by `spring.jpa.hibernate.ddl-auto=update`; Flyway + `validate` is **PLANNED**
Naming	Spring's physical naming strategy: camelCase fields become snake_case columns (`tenureMonths` → `tenure_months`)
Conventions	Surrogate `BIGINT AUTO_INCREMENT` keys (UUID for `users`); money `DECIMAL(18,2)`; enums stored as `VARCHAR` names; timestamps `DATETIME(6)` set in `@PrePersist`

1.1 Tables

Table	Entity	Purpose	Status
`users`	User	Login identity, role, verification flags	BUILT
`customers`	Customer	Personal profile and KYC of a user	BUILT
`accounts`	Account	Savings / current accounts and balances	BUILT
`transactions`	Transaction	Deposit / withdrawal / transfer headers	BUILT
`ledger_entries`	LedgerEntry	Double-entry DEBIT/CREDIT lines per account	BUILT
`beneficiaries`	Beneficiary	Saved payees of a customer	BUILT
`loans`	Loan	Loan contracts	BUILT
`emis`	Emi	Monthly installment schedule of a loan	BUILT
`audit_logs`	AuditLog	Who-did-what trail	BUILT
`cards`	Card	Debit / credit cards linked to accounts	PLANNED
`notification` (notification_db)	NotificationEntity	Notifications sent to users	BUILT
`flyway_schema_history`	—	Applied migrations	PLANNED

2. Entity Relationship Diagram


USERS
binary16 user_id PK
varchar full_name
varchar username UK
varchar email UK
varchar password
varchar phone_number UK
varchar role
varchar status
bit kyc_verified
bit email_verified
bit phone_verified
datetime last_login
datetime created_at
datetime updated_at
has profile
performed
CUSTOMERS
bigint id PK
binary16 user_id FK,UK
varchar first_name
varchar last_name
date date_of_birth
varchar phone_number UK
varchar address
varchar city
varchar state
varchar country
varchar postal_code
varchar kyc_status
varchar pan_number UK
varchar aadhaar_number UK
datetime created_at
AUDIT_LOGS
bigint id PK
binary16 performed_by FK
varchar action
varchar entity_name
varchar entity_id
longtext old_value
longtext new_value
varchar description
varchar ip_address
datetime created_at
owns
saves
borrows
ACCOUNTS
bigint id PK
varchar account_number UK
bigint customer_id FK
varchar account_holder_name
varchar branch
decimal balance
varchar account_type
varchar account_status
datetime created_at
datetime updated_at
bigint version
is payee in
receives disbursement
source of
destination of
linked to (planned)
has
BENEFICIARIES
bigint id PK
bigint customer_id FK
bigint beneficiary_account_id FK
varchar nick_name
varchar status
datetime created_at
LOANS
bigint id PK
bigint customer_id FK
bigint disbursement_account_id FK
varchar loan_type
decimal principal_amount
decimal interest_rate
int tenure_months
decimal emi_amount
varchar status
datetime created_at
datetime approved_at
datetime closed_at
TRANSACTIONS
bigint id PK
varchar transaction_reference UK
bigint source_account_id FK
bigint destination_account_id FK
decimal amount
varchar transaction_type
varchar transaction_status
decimal balance_after_transaction
varchar remarks
datetime created_at
CARDS
bigint id PK
bigint account_id FK
varchar card_number_masked
varchar card_type
varchar status
date expiry_date
repaid by
posts
EMIS
LEDGER_ENTRIES

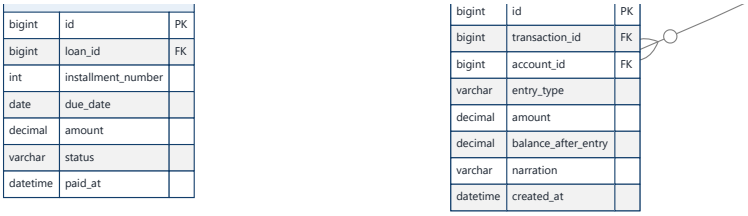


Figure 1 — banking_db ER diagram (crow's-foot). `CARDS` is planned.

2.1 Relationship summary

Relationship	Cardinality	FK column	JPA mapping
User → Customer	1 : 0..1	`customers.user_id` (unique)	`@OneToOne(LAZY)` on Customer
Customer → Account	1 : N	`accounts.customer_id`	`@ManyToOne(LAZY)` on Account
Account → Transaction (source)	0..1 : N	`transactions.source_account_id` (null for deposits)	`@ManyToOne(LAZY)`
Account → Transaction (destination)	0..1 : N	`transactions.destination_account_id` (null for withdrawals)	`@ManyToOne(LAZY)`
Transaction → LedgerEntry	1 : 1..2	`ledger_entries.transaction_id`	`@ManyToOne(LAZY)`
Account → LedgerEntry	1 : N	`ledger_entries.account_id`	`@ManyToOne(LAZY)`
Customer → Beneficiary	1 : N	`beneficiaries.customer_id`	`@ManyToOne(LAZY)`
Account → Beneficiary	1 : N	`beneficiaries.beneficiary_account_id`	`@ManyToOne(LAZY)`
Customer → Loan	1 : N	`loans.customer_id`	`@ManyToOne(LAZY)`
Account → Loan	1 : N	`loans.disbursement_account_id`	`@ManyToOne(LAZY)`
Loan → Emi	1 : 1..360	`emis.loan_id`	`@ManyToOne(LAZY)` on Emi (no collection on Loan)
User → AuditLog	0..1 : N	`audit_logs.performed_by` (null for anonymous / failed login)	`@ManyToOne(LAZY)`

2.2 notification_db

NOTIFICATION			
varchar36	notification_id	PK	
varchar64	user_id		banking users.user_id (logical ref)
varchar255	title		
varchar4000	content		
varchar16	notification_type		
varchar16	notification_priority		
varchar16	notification_status		
varchar128	idempotency_key	UK	
datetime	created_at		
datetime	updated_at		

Figure 2 — notification_db. `user_id` is a logical reference only (separate database, no FK).

3. Relational Schema (DDL)

The DDL below is the schema Hibernate generates from the entities. It is the intended content of the Flyway baseline migration `V1__baseline.sql` **PLANNED**.

```
CREATE TABLE users (
  user_id         BINARY(16)   NOT NULL PRIMARY KEY,
  full_name       VARCHAR(255) NOT NULL,
  username        VARCHAR(255) NOT NULL UNIQUE,
  email           VARCHAR(255) NOT NULL UNIQUE,
  password        VARCHAR(255) NOT NULL,              -- BCrypt hash
  phone_number    VARCHAR(255) NOT NULL UNIQUE,
  role            VARCHAR(255) NOT NULL,              -- CUSTOMER|MANAGER|EMPLOYEE|ADMIN|AUDITOR
  status          VARCHAR(255) NOT NULL,              -- ACTIVE|INACTIVE|BLOCKED|SUSPENDED
  kyc_verified    BIT          NOT NULL,
  email_verified  BIT          NOT NULL,
  phone_verified  BIT          NOT NULL,
  last_login      DATETIME(6),
  created_at      DATETIME(6)  NOT NULL,
  updated_at      DATETIME(6)
);

CREATE TABLE customers (
  id              BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id         BINARY(16)   NOT NULL UNIQUE,
  first_name      VARCHAR(255) NOT NULL,
  last_name       VARCHAR(255) NOT NULL,
  date_of_birth   DATE,
  phone_number    VARCHAR(15)  NOT NULL UNIQUE,
  address         VARCHAR(255) NOT NULL,
  city            VARCHAR(255) NOT NULL,
  state           VARCHAR(255) NOT NULL,
  country         VARCHAR(255) NOT NULL,
  postal_code     VARCHAR(255) NOT NULL,
  kyc_status      VARCHAR(255) NOT NULL,              -- PENDING|IN_PROGRESS|VERIFIED|REJECTED
  pan_number      VARCHAR(10)  UNIQUE,
  aadhaar_number  VARCHAR(12)  UNIQUE,
  created_at      DATETIME(6)  NOT NULL,
  CONSTRAINT fk_customer_user FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE accounts (
  id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
  account_number      VARCHAR(255)   NOT NULL UNIQUE,  -- "10" + 10 digits
  customer_id         BIGINT         NOT NULL,
  account_holder_name VARCHAR(255)   NOT NULL,
  branch              VARCHAR(255)   NOT NULL,
  balance             DECIMAL(18,2)  NOT NULL,
  account_type        VARCHAR(255)   NOT NULL,       -- SAVINGS|CURRENT
  account_status      VARCHAR(255)   NOT NULL,       -- ACTIVE|BLOCKED|FROZEN|CLOSED
  created_at          DATETIME(6)    NOT NULL,
  updated_at          DATETIME(6),
  version             BIGINT,                        -- optimistic lock
  CONSTRAINT fk_account_customer FOREIGN KEY (customer_id) REFERENCES customers(id)
);

CREATE TABLE transactions (
  id                        BIGINT AUTO_INCREMENT PRIMARY KEY,
  transaction_reference     VARCHAR(255)  NOT NULL UNIQUE,  -- "TXN" + 13 digits
  source_account_id         BIGINT,
  destination_account_id    BIGINT,
  amount                    DECIMAL(18,2) NOT NULL,
  transaction_type          VARCHAR(255)  NOT NULL,   -- DEPOSIT|WITHDRAWAL|TRANSFER
  transaction_status        VARCHAR(255)  NOT NULL,   -- PENDING|SUCCESS|FAILED|REVERSED
  balance_after_transaction DECIMAL(18,2) NOT NULL,
  remarks                   VARCHAR(255),
  created_at                DATETIME(6)   NOT NULL,
  CONSTRAINT fk_txn_source FOREIGN KEY (source_account_id)      REFERENCES accounts(id),
```

```
  CONSTRAINT fk_txn_dest   FOREIGN KEY (destination_account_id) REFERENCES accounts(id)
);

CREATE TABLE ledger_entries (
  id                   BIGINT AUTO_INCREMENT PRIMARY KEY,
  transaction_id       BIGINT        NOT NULL,
  account_id           BIGINT        NOT NULL,
  entry_type           VARCHAR(255)  NOT NULL,         -- DEBIT|CREDIT
  amount               DECIMAL(18,2) NOT NULL,
  balance_after_entry  DECIMAL(18,2) NOT NULL,
  narration            VARCHAR(255),
  created_at           DATETIME(6)   NOT NULL,
  CONSTRAINT fk_ledger_txn     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
  CONSTRAINT fk_ledger_account FOREIGN KEY (account_id)     REFERENCES accounts(id)
);

CREATE TABLE beneficiaries (
  id                      BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_id             BIGINT       NOT NULL,
  beneficiary_account_id  BIGINT       NOT NULL,
  nick_name               VARCHAR(255) NOT NULL,
  status                  VARCHAR(255) NOT NULL,       -- PENDING|ACTIVE|BLOCKED
  created_at              DATETIME(6)  NOT NULL,
  CONSTRAINT fk_benef_customer FOREIGN KEY (customer_id)           REFERENCES customers(id),
  CONSTRAINT fk_benef_account  FOREIGN KEY (beneficiary_account_id) REFERENCES accounts(id)
);

CREATE TABLE loans (
  id                       BIGINT AUTO_INCREMENT PRIMARY KEY,
  customer_id              BIGINT        NOT NULL,
  disbursement_account_id  BIGINT        NOT NULL,
  loan_type                VARCHAR(255)  NOT NULL,     -- PERSONAL|HOME|VEHICLE|EDUCATION
  principal_amount         DECIMAL(18,2) NOT NULL,
  interest_rate            DECIMAL(5,2)  NOT NULL,     -- annual %, e.g. 10.50
  tenure_months            INT           NOT NULL,
  emi_amount               DECIMAL(18,2) NOT NULL,
  status                   VARCHAR(255)  NOT NULL,     -- PENDING|ACTIVE|REJECTED|CLOSED
  created_at               DATETIME(6)   NOT NULL,
  approved_at              DATETIME(6),
  closed_at                DATETIME(6),
  CONSTRAINT fk_loan_customer FOREIGN KEY (customer_id)            REFERENCES customers(id),
  CONSTRAINT fk_loan_account  FOREIGN KEY (disbursement_account_id) REFERENCES accounts(id)
);

CREATE TABLE emis (
  id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
  loan_id             BIGINT        NOT NULL,
  installment_number  INT           NOT NULL,
  due_date            DATE          NOT NULL,
  amount              DECIMAL(18,2) NOT NULL,
  status              VARCHAR(255)  NOT NULL,          -- PENDING|PAID|OVERDUE
  paid_at             DATETIME(6),
  CONSTRAINT fk_emi_loan FOREIGN KEY (loan_id) REFERENCES loans(id)
);

CREATE TABLE audit_logs (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  performed_by  BINARY(16),
  action        VARCHAR(255) NOT NULL,   -- CREATE|UPDATE|DELETE|LOGIN|LOGOUT|LOGIN_FAILED
  entity_name   VARCHAR(255) NOT NULL,
  entity_id     VARCHAR(255),
  old_value     LONGTEXT,
  new_value     LONGTEXT,
  description   VARCHAR(255),
```

```
  ip_address    VARCHAR(64),
  created_at    DATETIME(6)  NOT NULL,
  CONSTRAINT fk_audit_user FOREIGN KEY (performed_by) REFERENCES users(user_id)
);
```

4. Data Dictionary

Column-level reference. **Req** = NOT NULL. Validation refers to the request-DTO rules that guard the column at the API boundary.

4.1 users

Column	Type	Req	Description / rules
user_id	UUID	PK	Generated by Hibernate (`GenerationType.UUID`)
full_name	varchar	Y	3–100 chars
username	varchar	Y, UK	3–50 chars
email	varchar	Y, UK	Valid e-mail; login identifier and JWT subject
password	varchar	Y	BCrypt hash (raw ≥ 8 chars); never returned or logged
phone_number	varchar	Y, UK	Indian mobile: `^[6-9]\d{9}$`
role	enum	Y	Defaults to `CUSTOMER` on register
status	enum	Y	Defaults to `ACTIVE`
kyc_verified	bool	Y	Set true when customer KYC is completed; false when customer deleted
email_verified / phone_verified	bool	Y	Always false today (verification flow PLANNED)
last_login	datetime	N	Not yet updated on login GAP
created_at / updated_at	datetime	Y / N	Set by service on register

4.2 customers

Column	Type	Req	Description / rules
id	bigint	PK	Auto-increment
user_id	UUID	Y, UK, FK	One customer profile per user; resolved from request `email`
first_name / last_name	varchar	Y	3–50 chars on create, 2–50 on update
date_of_birth	date	N (API: required)	ISO `yyyy-MM-dd`
phone_number	varchar(15)	Y, UK	`^[6-9]\d{9}$` ; uniqueness also checked in service
address	varchar	Y	≤ 255
city / state / country	varchar	Y	≤ 100 each
postal_code	varchar	Y	6 digits: `^\d{6}$`
kyc_status	enum	Y	PENDING on create → VERIFIED on KYC completion
pan_number	varchar(10)	N, UK	`^[A-Z]{5}[0-9]{4}[A-Z]$` **(sensitive)**
aadhaar_number	varchar(12)	N, UK	`^\d{12}$` **(sensitive)**
created_at	datetime	Y	`@PrePersist`

4.3 accounts

Column	Type	Req	Description / rules
id	bigint	PK	Auto-increment
account_number	varchar	Y, UK	System-generated: `"10"` + 10 random digits (SecureRandom), regenerated on collision
customer_id	bigint	Y, FK	Owner
account_holder_name	varchar	Y	Copied from customer first + last name
branch	varchar	Y	≤ 100 chars
balance	decimal(18,2)	Y	Opening balance ≥ 0; never negative (withdraw/transfer check)
account_type	enum	Y	SAVINGS | CURRENT
account_status	enum	Y	ACTIVE default; only ACTIVE accounts can transact
created_at / updated_at	datetime	Y / N	`@PrePersist` / `@PreUpdate`
version	bigint	N	JPA `@Version` — optimistic locking

4.4 transactions

Column	Type	Req	Description / rules
id	bigint	PK	
transaction_reference	varchar	Y, UK, immutable	`"TXN"` + 13 digits
source_account_id	bigint	N, FK	Set for WITHDRAWAL and TRANSFER
destination_account_id	bigint	N, FK	Set for DEPOSIT and TRANSFER
amount	decimal(18,2)	Y	≥ 1.00
transaction_type	enum	Y	DEPOSIT | WITHDRAWAL | TRANSFER
transaction_status	enum	Y	Written as SUCCESS (failures roll back, nothing stored); FAILED / REVERSED reserved
balance_after_transaction	decimal(18,2)	Y	Account balance after posting (source balance for transfers)
remarks	varchar(255)	N	Free text from client
created_at	datetime	Y	Immutable

4.5 ledger_entries

Column	Type	Req	Description / rules
transaction_id	bigint	Y, FK	Parent transaction
account_id	bigint	Y, FK	Account whose balance moved
entry_type	enum	Y	DEBIT (money out) | CREDIT (money in)

amount	decimal(18,2)	Y	Always positive
balance_after_entry	decimal(18,2)	Y	Running balance — statement line
narration	varchar(255)	N	Remarks, or `"<TYPE> <reference>"`

4.6 beneficiaries

Column	Type	Req	Description / rules
customer_id	bigint	Y, FK	Owner of the payee list
beneficiary_account_id	bigint	Y, FK	Payee account; (customer_id, beneficiary_account_id) must be unique (service check)
nick_name	varchar	Y	≤ 100 chars
status	enum	Y	PENDING on add → ACTIVE on activation; only ACTIVE can receive transfers

4.7 loans and emis

Column	Type	Req	Description / rules
loans.loan_type	enum	Y	PERSONAL | HOME | VEHICLE | EDUCATION
loans.principal_amount	decimal(18,2)	Y	≥ 1000.00
loans.interest_rate	decimal(5,2)	Y	Annual %, ≥ 0.01
loans.tenure_months	int	Y	1–360
loans.emi_amount	decimal(18,2)	Y	Computed at application: `P·r·(1+r)^n / ((1+r)^n – 1)`, r = rate/12/100, HALF_UP to 2 dp
loans.status	enum	Y	PENDING → ACTIVE / REJECTED; ACTIVE → CLOSED
loans.approved_at / closed_at	datetime	N	Set on approve / last EMI paid
emis.installment_number	int	Y	1..tenure_months
emis.due_date	date	Y	Approval date + n months
emis.amount	decimal(18,2)	Y	= loans.emi_amount
emis.status	enum	Y	PENDING → PAID; OVERDUE via scheduled job **PLANNED**
emis.paid_at	datetime	N	Set on payment

4.8 audit_logs

Column	Type	Req	Description / rules
performed_by	UUID	N, FK	Authenticated user from SecurityContext; null for anonymous events
action	enum	Y	CREATE | UPDATE | DELETE | LOGIN | LOGOUT | LOGIN_FAILED
entity_name / entity_id	varchar	Y / N	e.g. `"Account"`, `"42"`
old_value / new_value	LONGTEXT	N	Short `key=value` snapshots; never contain passwords
description	varchar(255)	N	Human readable, e.g. “Account frozen”

ip_address	varchar(64)	N	Client IP (first `X-Forwarded-For` value or remote address)

4.9 Enumerations

Enum	Values
Role	CUSTOMER, MANAGER, EMPLOYEE, ADMIN, AUDITOR
UserStatus	ACTIVE, INACTIVE, BLOCKED, SUSPENDED
KYCStatus	PENDING, IN_PROGRESS, VERIFIED, REJECTED
AccountType	SAVINGS, CURRENT
AccountStatus	ACTIVE, BLOCKED, FROZEN, CLOSED
TransactionType	DEPOSIT, WITHDRAWAL, TRANSFER
TransactionStatus	PENDING, SUCCESS, FAILED, REVERSED
EntryType	DEBIT, CREDIT
BeneficiaryStatus	ACTIVE, BLOCKED, PENDING
LoanType	PERSONAL, HOME, VEHICLE, EDUCATION
LoanStatus	ACTIVE, PENDING, REJECTED, CLOSED
EmiStatus	PENDING, PAID, OVERDUE
AuditAction	CREATE, UPDATE, DELETE, LOGIN, LOGOUT, LOGIN_FAILED
NotificationType / Priority / Status	EMAIL, SMS, PUSH, IN_APP · LOW, MEDIUM, HIGH · CREATED, QUEUED, PROCESSING, SENT, FAILED, RETRYING

5. Constraints and Integrity Rules

5.1 Database-enforced

Type	Constraint
Primary keys	All tables (UUID for users, identity BIGINT elsewhere, VARCHAR(36) for notification)
Unique	users.username, users.email, users.phone_number, customers.user_id, customers.phone_number, customers.pan_number, customers.aadhaar_number, accounts.account_number, transactions.transaction_reference, notification.idempotency_key
Foreign keys	All 13 relationships in section 2.1 (no `ON DELETE CASCADE` ; deleting a parent with children fails)
NOT NULL	As per DDL in section 3
Optimistic lock	accounts.version incremented on every update

5.2 Application-enforced (service layer)

- Balance never goes negative: withdraw/transfer fail with *insufficient balance*.
- Only `ACTIVE` accounts can be source or destination of a transaction.
- A transfer cannot have the same source and destination; exactly one of `destinationAccountId` / `beneficiaryId` must be given.
- Transfers via beneficiary require status ACTIVE and the beneficiary must belong to the source account's customer.
- An account must have zero balance to be closed; a CLOSED account cannot be modified.
- A customer cannot add the same account as a beneficiary twice.
- A loan can be approved or rejected only while PENDING; an EMI cannot be paid twice.
- PAN / Aadhaar / phone may not be linked to another customer.
- Double-entry invariant: for every TRANSFER, Σ DEBIT = Σ CREDIT (verified by `AccountingLogicTest`).

6. Indexes

Index	Table(columns)	Source	Serves	Status
PRIMARY	every table (id)	PK	findById	BUILT
UK_* (auto)	all unique columns listed in 5.1	`unique = true`	findByEmail, findByAccountNumber, findByTransactionReference, findByPanNumber ...	BUILT
FK_* (auto)	every FK column	InnoDB creates an index for each FK	findByCustomerId, findByLoanId..., findByAccountId...	BUILT
idx_txn_source	transactions(source_account_id)	V2__indexes.sql	Transaction history	PLANNED
idx_txn_dest	transactions(destination_account_id)	V2__indexes.sql	Transaction history	PLANNED
idx_ledger_account_created	ledger_entries(account_id, created_at)	V2__indexes.sql	Ledger by account ordered by date	PLANNED
idx_audit_entity	audit_logs(entity_name, entity_id, created_at)	V2__indexes.sql	Audit by entity	PLANNED
idx_emi_loan_installment	emis(loan_id, installment_number)	V2__indexes.sql	EMI schedule ordered	PLANNED
idx_notification_user_id	notification(user_id)	@Index	getByUserId	BUILT

```
-- V2__indexes.sql (planned)
CREATE INDEX idx_txn_source            ON transactions(source_account_id);
CREATE INDEX idx_txn_dest              ON transactions(destination_account_id);
CREATE INDEX idx_ledger_account_created ON ledger_entries(account_id, created_at);
CREATE INDEX idx_audit_entity          ON audit_logs(entity_name, entity_id, created_at);
CREATE INDEX idx_emi_loan_installment  ON emis(loan_id, installment_number);
```

7. Migration History

7.1 Schema evolution so far (Hibernate `ddl-auto=update`)

Stage	Change	Tables affected
1	Authentication: User entity with UUID key, roles, status	users
2	Customer profile + KYC fields	customers
3	Accounts with generated numbers, `@Version` optimistic locking	accounts
4	Transactions (deposit, withdraw, transfer)	transactions
5	Double-entry ledger	ledger_entries
6	Audit logging	audit_logs
7	Beneficiaries (+ transfer via beneficiary)	beneficiaries
8	Embedded notification module removed — moved to notification-service	(notification tables moved to notification_db)
9	Loans and EMI schedule	loans, emis

7.2 Flyway plan PLANNED

Version	File	Content
V1	`V1__baseline.sql`	Full schema of section 3 (captured with `mysqldump --no-data banking_db`)
V2	`V2__indexes.sql`	Indexes from section 6
V3	`V3__cards.sql`	`cards` table when the Card module is implemented
V4+	`V<n>__<description>.sql`	Every future schema change; never edit an applied migration

> Procedure: add `flyway-mysql`, place files in `src/main/resources/db/migration`, set `spring.jpa.hibernate.ddl-auto=validate` (prod profile). On an existing database use `spring.flyway.baseline-on-migrate=true` so V1 is marked as applied rather than re-run.

8. Data Retention and Backup

Data	Policy
transactions, ledger_entries	Append-only; never updated or deleted (financial record). Corrections are new reversing transactions (REVERSED status reserved).
audit_logs	Append-only; retain ≥ 8 years for compliance; read access for AUDITOR / ADMIN only (planned).
customers	Hard delete exists today (`DELETE /customers/{id}`); fails if accounts reference the customer. Soft delete is recommended **PLANNED** .
Backups	See document 07, section “Backup and recovery”.